COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

03

Lab and problem workbook

Practice, integration and assessment

20 practical briefs · 75 written problems · 25 review questions · 10 algebra-bridge items

PACKAGE 03 / EXPANDED VOLUME 02 / SEPTEMBER 28, 2026

Self-study: complete the derivations, implementation, testing and review.

Contents

PDF bookmarks also navigate the lessons, quizzes, assignments and references.

Lab and problem workbook 4

Module 06 — Discrete mathematical reasoning 6

Lab 06.1 — Algebra, functions, logarithms and trigonometry . . . 6

Lab 06.2 — Logic, sets and quantified claims . . . 6

Lab 06.3 — Proofs, induction and invariants . . . 7

Lab 06.4 — Counting, graphs and recurrences . . . 8

Module 06 — Fifteen written problems . . . 9

Module 06 — Review quiz . . . 10

Module 07 — Linear algebra and game geometry 11

Lab 07.1 — Vectors, dot products and projections . . . 11

Lab 07.2 — Matrices, bases and linear systems . . . 11

Lab 07.3 — Homogeneous transforms, cameras and quaternions . . . 12

Lab 07.4 — Eigenvalues, least squares and singular values . . . 13

Module 07 — Fifteen written problems . . . 14

Module 07 — Review quiz . . . 15

Module 08 — Calculus, mechanics and numerical simulation 16

Lab 08.1 — Limits, derivatives and rates . . . 16

Lab 08.2 — Integrals, accumulation and continuous probability . . . 16

Lab 08.3 — Newtonian mechanics and differential equations . . . 17

Lab 08.4 — Time integration, stability and error . . . 18

Module 08 — Fifteen written problems 19

Module 08 — Review quiz 20

Module 09 — Probability, statistics and evidence 21

Lab 09.1 — Probability and conditional reasoning 21

Lab 09.2 — Random variables, distributions and sampling 21

Lab 09.3 — Inference, uncertainty and benchmark design 22

Lab 09.4 — Regression, correlation and ethical telemetry 23

Module 09 — Fifteen written problems 24

Module 09 — Review quiz 25

Module 10 — Signals, optimization and numerical foundations 26

Lab 10.1 — Multivariable calculus and gradients 26

Lab 10.2 — Sampling, aliasing and Fourier ideas 26

Lab 10.3 — Convolution, filters and systems 27

Lab 10.4 — Optimization, conditioning and approximation budgets 28

Module 10 — Fifteen written problems 29

Module 10 — Review quiz 30

Lab and problem workbook

Use this workbook with the Student Handbook. Every lesson has a full practical assignment plus a computational slice with a supplied checker. Complete both the written/model work and the coding slice. Checkers do not grade proofs, coordinate explanations or statistical validity.

Assessment and independent work

Each of the five lesson questions is scored 0-2: zero for absent/incorrect; one for a partly correct answer or a correct number without requested reasoning; two for a correct answer with the relevant argument. **Keep the original three-question threshold: questions 1-3 need at least 5/6. The full five-question quiz also needs at least 8/10.** Complete assigned study, the full practical assignment and the unaided transfer task before marking the lesson complete. Code-checker success alone is not completion.

The five module-review questions use the same two-point scale; suggested pass is 8/10. Correct missed work and solve a changed example. For the 15 written problems in each module, retain an independent attempt and a correction; proofs must identify assumptions and justified steps. At least two derivations/proofs per module must be reviewed, not merely checked for a final number.

The Gate B project uses the original 100-point rubric: correctness/requirements 30; tests/verification 25; reasoning/tradeoffs 15; quantitative error/performance analysis 10; reproducibility 10; communication/provenance 10. Suggested pass is 80/100, every essential check completed, and no unresolved critical defect in the claimed scope. A reviewer may accept another technically sound method when its assumptions and evidence are explicit.

Reference answers are study aids after an attempt. Explain all assisted code, record help received, and do the transfer task without following a supplied solution. Nothing here grants an academic credential. Passing a few numerical examples does not prove a mathematical theorem or demonstrate all of a semester's subject depth.

Algebra bridge — ten readiness questions

These support the original 06.1 algebra requirement. Work them independently; correct gaps with the first lesson and R40 before applying the formulas in later labs.

A01. Solve 3x+2=14.

A02. Solve 5-2x=11.

A03. Solve x/4+3=8.

A04. Solve 2(x-3)=10.

A05. Solve (x+1)/3=(x-1)/2.

A06. Solve x^2=9 over the real numbers.

A07. Solve x(x-4)=0.

A08. Solve 2^x=32.

A09. Solve sqrt(x-1)=3 and state the real domain.

A10. Solve d=v*t for t when d=18 m and v=3 m/s. State the forbidden denominator.

Common coding contract

Unless a lesson says otherwise, numeric inputs are finite built-in int/float values, not Boolean values; sequences are lists or tuples. Domain/type/dimension errors use ValueError. Nonfinite arithmetic is an error, not a successful result. These small teaching routines are not designed to handle every extreme-scale numerical problem; rescale and analyze inputs, and do not promise arbitrary-range accuracy. Keep your own tests separate from supplied checks.

Run `python check_lab.py 07.1` for one student exercise; use `--reference` only to check the supplied implementation. Running `--all` on untouched student files is expected to fail. The 20 code exercises are a subset of the 20 full practical briefs, not an extra 20 major projects.

Module 06 — Discrete mathematical reasoning

Prerequisites: Volume 01; complete the algebra diagnostic in START_HERE before proceeding.

Lab 06.1 — Algebra, functions, logarithms and trigonometry

Original brief: Solve distance = speed*time for each variable and state forbidden inputs. Convert 0, 90, 180 and 360 degrees to radians. Pass: solve ten algebra problems and explain logarithms using repeated halving before advancing.

Student file: `labs/student/lab_06_1.py`

Functions: `travel_time`

Coding contract. Return distance/speed. Distance is finite and nonnegative; speed is finite and strictly positive. Inputs use consistent units. Reject booleans, text, nonfinite values, negative distance and zero/negative speed with ValueError. Zero distance returns zero. A path length is an input, not something this function finds.

1. Implement travel_time(distance, speed), requiring finite nonnegative distance and finite positive speed. Return seconds for inputs in meters and meters/second.
2. Complete the original ten algebra problems; the readiness bridge supplies concrete practice. Show all rearrangements and domain restrictions.
3. Tabulate directions at 0, 90, 180 and 360 degrees. Explain near-zero floating-point components rather than forcing all small numbers to zero.
4. Compare floor-halving and ceiling-halving for sizes 1, 2, 3, 1000 and 1024. Record the chosen recurrence.

Check command: `python check_lab.py 06.1`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 06.2 — Logic, sets and quantified claims

Original brief: Formalize five requirements for the grid game using sets and quantifiers. Construct truth tables for a door-access rule. Pass: provide a counterexample or witness for each negated requirement.

Student file: `labs/student/lab_06_2.py`

Functions: `implies`

Coding contract. Accept exactly two Boolean operands. Return material implication, which is false only for True -> False. Reject integer 0/1 and other non-Boolean inputs with ValueError; Python truthiness is not the specified interface.

1. Implement implies(p, q) for actual Boolean inputs, rejecting integer lookalikes under this exercise contract.
2. Write five quantified requirements for Training Yard or a small fictional unit world. State each domain explicitly.
3. Construct the eight-row key/level/stamina predicate table, then test the numeric boundaries independently.
4. For each requirement, provide a witness or counterexample for its negation. Include an empty collection and a repeated ID.

Check command: `python check_lab.py 06.2`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 06.3 — Proofs, induction and invariants

Original brief: Prove the formula 1+...+n = n(n+1)/2 by induction. Then write an invariant for counting living units in a prefix of a list. Pass: show initialization, preservation and termination explicitly.

Student file: `labs/student/lab_06_3.py`

Functions: `count_positive_prefix`

Coding contract. Accept a list/tuple of finite numbers and an integer prefix length k with 0 <= k <= len(values). Return the number of strictly positive entries in values[:k]. Validate all input entries, even entries after the selected prefix. An empty list with k=0 returns zero. Reject bool for k and invalid ranges. The written invariant and termination argument are still required.

1. Implement count_positive_prefix(values, k); allow an empty list with k=0, reject nonfinite data and invalid k, and do not modify inputs.
2. Write the full induction proof for 1+...+n, including n=0 and the algebraic step.
3. Write initialization, preservation, exit consequence, and a decreasing termination measure for the prefix-count loop.
4. Insert an off-by-one or >= defect in a disposable copy; find a counterexample and explain which proof-to-code correspondence fails.

Check command: `python check_lab.py 06.3`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 06.4 — Counting, graphs and recurrences

Original brief: Build a dependency graph for game startup and identify an invalid cycle. Count loadout combinations with and without a restriction. Pass: explain the recurrence for halving a search range and distinguish depth from total work.

Student file: `labs/student/lab_06_4.py`

Functions: `halving_steps`

Coding contract. Accept a strictly positive built-in integer n, not bool. Repeatedly replace n with n//2 until n<=1; return the number of replacements. The result is floor(log2(n)), but implement the integer loop and prove its termination. Reject zero, negative and noninteger inputs. The graph-model assignment is a separate part of this lesson.

1. Implement halving_steps(n) using repeated floor division, accepting positive integers only. Do not implement it by rounding a floating logarithm.
2. Draw a startup graph with at least six nodes, label edge meaning, give a valid order, then add an explicit invalid cycle.
3. Count the four-slot loadouts with and without the stated restriction and check by enumeration.
4. Derive both a one-branch halving recurrence and a two-branch divide-and-combine recurrence. Distinguish comparisons, depth, and elapsed time.

Check command: `python check_lab.py 06.4`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Module 06 — Fifteen written problems

Show intermediate reasoning, domains and units. Problems marked as proofs/derivations require an argument, not only an answer. Preserve wrong attempts and corrections.

06.P01 — calculation

A unit moves at 12 m/s for 0.25 s. Find distance, then solve for the time needed to travel 15 m at the same constant speed.

06.P02 — domain

State the real domains of sqrt(9-x), 1/(x+2), and log2(x-1).

06.P03 — calculation

Convert 30, 135, and -90 degrees to radians. Give the unit direction at -90 degrees in a y-up plane.

06.P04 — calculation

Compute log2(256), log10(1000), and the floor-halving count for 1000. Explain why the last is not rounded log2(1000).

06.P05 — logic

Construct the truth table for P -> Q and for its converse. Which rows distinguish them?

06.P06 — logic

Negate: every living unit has some valid owner. Keep the living-unit domain explicit.

06.P07 — proof

Prove that the sum of the first n odd positive integers is n^2 for n>=0.

06.P08 — proof

Prove that the sum of two odd integers is even. State your domain.

06.P09 — counterexample

Give a counterexample to each claim: every positive number is an integer; (a+b)^2=a^2+b^2 for all real a,b; a successful test proves all inputs correct.

06.P10 — counting

A game offers 4 character classes, 3 weapons and 2 maps. One class cannot use one weapon on either map; all other choices are allowed. Count valid combinations.

06.P11 — counting

In a collection, 12 items are red, 9 are heavy, and 4 are both. How many are red or heavy?

06.P12 — graph

For a simple undirected graph on 8 vertices, find the maximum number of edges. Contrast directed edges without self-loops.

06.P13 — graph

Edges A->C, B->C, C->D, B->E mean prerequisites. Give two valid startup orders and one edge that creates a cycle.

06.P14 — derivation

For n=2^k, solve T(n)=T(n/2)+1 with T(1)=1.

06.P15 — derivation

For n=2^k, solve T(n)=2T(n/2)+n with T(1)=1.

Module 06 — Review quiz

Five questions, two points each. Answer without the key; suggested pass is 8/10 with reasoning.

06.R1. A new door opens with a key, or with both level>=7 and energy>=4. State a Boolean rule and the numeric boundary tests.

06.R2. Explain why an invariant and a decreasing variant have different roles.

06.R3. A log2 estimate says 9.97 steps. What must be specified before turning that into an integer loop count?

06.R4. A startup graph has two independent prerequisites for a scene. Must there be only one valid order?

06.R5. What evidence distinguishes a tested general claim from a proved one?

Integration project — A mathematical specification notebook

A versioned document plus scripts containing five quantified world rules, complete door truth tables, two proofs, a six-node dependency graph, two recurrence analyses, and three counterexamples.

Pass evidence: The proof reviewer can identify initialization/preservation/termination where applicable. Every graph edge has a stated meaning; cyclic input is rejected. The ten algebra-bridge items and all 15 module problems have independent attempts and corrections.

Reviewer's changed requirement: Raise the access threshold, change a counting restriction, and replace floor-halving by ceiling-halving. Explain what changes without rewriting unrelated assumptions.

Retain the artifact for Gate B; this work accumulates rather than being discarded after the module.

Module 07 — Linear algebra and game geometry

Prerequisites: Module 06.1; basic Python functions.

Lab 07.1 — Vectors, dot products and projections

Original brief: Implement length, dot and safe normalization. Use dot products to determine whether a target is in front of a character. Pass: test zero vectors, perpendicular vectors and opposite directions.

Student file: `labs/student/lab_07_1.py`

Functions: `dot`, `normalize`

Coding contract. dot accepts two nonempty same-length list/tuple vectors of finite numbers and returns the scalar inner product. normalize accepts a nonempty finite vector, rejects the zero vector, and returns a tuple of the same dimension with unit Euclidean length within tolerance. Scaling before computing the length avoids direct squaring overflow in extreme-magnitude cases. Preserve signs. Reject incompatible dimensions and bool components.

1. Implement dot(a,b) and normalize(v), documenting finite inputs, equal dimensions, and a deliberate zero-vector policy. Do not mutate the arguments.
2. Test perpendicular, opposite, and identical directions; include zero and extreme finite magnitudes in normalization.
3. Use target-position, not the target point alone, in a front-facing query. Decide what happens when target equals position.
4. Prove that the projection residual is perpendicular to a nonzero projection direction, then check a numerical case.

Check command: `python check_lab.py 07.1`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 07.2 — Matrices, bases and linear systems

Original brief: Implement 2-by-2 multiplication and solve two small linear systems by elimination on paper. Pass: show a pair of transforms that do not commute and explain a singular example geometrically.

Student file: `labs/student/lab_07_2.py`

Functions: `matmul`, `solve2`

Coding contract. matmul accepts nonempty rectangular list/tuple matrices with matching inner dimensions; returns tuple rows. Reject ragged or incompatible matrices. solve2 accepts a 2x2 matrix and two-entry right-hand side; returns (x,y). Its relative pivot threshold rtol must satisfy 0<=rtol<1. Use scaled elimination and row pivoting; reject singular or numerically small pivots. A rejected small pivot is a policy result, not a general condition-number certificate.

1. Implement matmul(a,b) and solve2(a,b), with dimension checks and documented singular/small-pivot rejection. Test a row-swap case.
2. Show the rotation/scale noncommutativity example entirely on paper before checking it in code.
3. Solve two small linear systems by elimination and distinguish a unique, inconsistent, and underdetermined case.
4. Provide a nonzero nullspace vector for a singular matrix and verify that it explains two inputs with the same output.

Check command: `python check_lab.py 07.2`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 07.3 — Homogeneous transforms, cameras and quaternions

Original brief: Create a parented object and camera using documented conventions. Pass: test identity, inverse round trips within tolerance, a translated direction that remains unchanged, and a parent rotation that moves its child correctly.

Student file: `labs/student/lab_07_3.py`

Functions: `transform4`

Coding contract. Multiply a finite 4x4 matrix by a four-component column vector and return a four-tuple. Do not perform a perspective divide or force w to one. Points use w=1, directions w=0 in the chosen affine convention. Test a non-affine last row to detect an accidental divide. Scene hierarchy, camera inverse and quaternion explanations remain written/integration tasks.

1. Implement transform4(m,v) for a 4x4 matrix and a four-component column vector. It must not perform perspective division.
2. Create the parent-child chain above, a camera inverse round trip, and an independently derived quarter-turn test.
3. Verify points and directions have different translation behavior. Write an explicit grid-to-world conversion for y-down versus y-up.
4. Derive one axis-angle quaternion and compare its rotation with a matrix; check q and -q and reject the zero quaternion.

Check command: `python check_lab.py 07.3`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 07.4 — Eigenvalues, least squares and singular values

Original brief: Fit a line to five noisy position/time observations and report residuals. Analyze repeated multiplication by a diagonal matrix. Pass: distinguish data noise, poor conditioning and a programming error in your report.

Student file: `labs/student/lab_07_4.py`

Functions: `fit_line`

Coding contract. Fit y=a*x+b by ordinary unweighted least squares with an intercept. Accept at least two finite paired observations in equal-length lists/tuples. Return (slope,intercept). Reject predictors with no resolvable variation. Use centered sums rather than explicitly forming normal-equation inverses. A small training residual does not guarantee valid extrapolation or well-conditioned data.

1. Implement fit_line(xs,ys), returning slope then intercept, and reject unequal lengths, fewer than two pairs, nonfinite data, and a constant predictor.
2. Fit the five supplied measurements and report every residual, their sum, and the squared-error objective.
3. Analyze powers of two diagonal matrices symbolically and compare with code; include a negative eigenvalue.
4. Construct a diagonal sensitivity example and distinguish noise, conditioning, and a deliberately introduced indexing defect.

Check command: `python check_lab.py 07.4`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Module 07 — Fifteen written problems

Show intermediate reasoning, domains and units. Problems marked as proofs/derivations require an argument, not only an answer. Preserve wrong attempts and corrections.

07.P01 — calculation

Find displacement and distance from (2,-1,3) to (5,3,3).

07.P02 — calculation

Normalize (-6,8) and check its squared length.

07.P03 — calculation

Project v=(4,2) onto b=(1,1). Find the perpendicular residual.

07.P04 — geometry

For unit facing (1,0), compare displacements (1,1),(-1,1),(0,2) against an inclusive 45-degree field of view.

07.P05 — proof

Prove that v-((v·b)/(b·b))b is perpendicular to nonzero b.

07.P06 — proof

Prove that an orthonormal matrix Q preserves Euclidean squared length.

07.P07 — calculation

Multiply [[1,2],[3,4]] by [[2,0],[1,2]].

07.P08 — calculation

Solve x+2y=5 and 3x-y=1 by elimination.

07.P09 — geometry

Give a nonzero nullspace vector of [[1,2],[2,4]], and state its rank.

07.P10 — transform

For column vectors, apply child translation (3,0,0) under parent T(5,1,0)*Rz(pi/2). Find the child origin in world space.

07.P11 — transform

A no-rotation camera sits at (2,3,4). Find the view coordinates of world point (5,7,10), and of direction (1,0,0).

07.P12 — derivation

Derive slope and intercept for ordinary least squares with an intercept.

07.P13 — quaternion

Find the scalar-first quaternion for a 180-degree rotation around +z. Rotate (1,0,0).

07.P14 — singular values

For A=[[3,0],[4,0]], find the singular values and rank using A^T A.

07.P15 — conditioning

For A=diag(1,10^-6), compare solutions for b=(1,0) and b_changed=(1,10^-6). What is the two-norm condition number?

Module 07 — Review quiz

Five questions, two points each. Answer without the key; suggested pass is 8/10 with reasoning.

07.R1. How can an inverse round-trip test pass despite a convention error?

07.R2. When should normalization fail instead of inventing a direction?

07.R3. What does a nonzero vector in the nullspace tell you geometrically?

07.R4. Explain the difference among residual, forward error, and conditioning.

07.R5. A child is positioned by M_parent*M_child. Why cannot the factors be swapped casually?

Integration project — A tested geometry and fitting toolkit

A small vector/matrix implementation with tests, a parent-child scene calculation, a camera coordinate conversion, one quaternion/matrix agreement case, and a five-point least-squares report.

Pass evidence: Include zero-vector rejection, noncommuting transforms, a singular system, known hand-derived transforms, q/-q agreement, and residual/conditioning distinctions. Record column-vector order, handedness, axes, and homogeneous point/direction rules.

Reviewer's changed requirement: Convert the scene to an explicitly y-down interface without silently changing the world-space math. Ask the reviewer to choose a new parent rotation and predict the child location.

Retain the artifact for Gate B; this work accumulates rather than being discarded after the module.

Module 08 — Calculus, mechanics and numerical simulation

Prerequisites: Modules 06.1 and 07.1; work through the linked calculus problem sets.

Lab 08.1 — Limits, derivatives and rates

Original brief: Compare analytic and finite-difference derivatives of three functions over several step sizes. Pass: include units, an error table and an explanation of why the smallest tested h need not be best.

Student file: `labs/student/lab_08_1.py`

Functions: `centered_difference`

Coding contract. Return (f(x+h)-f(x-h))/(2*h). Require finite x and finite strictly positive h; reject a perturbation for which x+h or x-h is indistinguishable from x. Validate finite function results. A failing tiny step should not silently report a derivative of zero. Analyze the dependence of error on h separately.

1. Implement centered_difference(f,x,h), requiring finite inputs, positive resolvable h, and finite function evaluations.
2. Compare analytic and numerical derivatives for t^2, t^3, and sin(t), using at least five step sizes. State radians for the sine argument.
3. Include the corner |t| at zero and explain why a centered value of zero does not establish differentiability.
4. Record absolute errors, units, and the approximation/rounding tradeoff. Do not smooth away a failed case without documenting it.

Check command: `python check_lab.py 08.1`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 08.2 — Integrals, accumulation and continuous probability

Original brief: Implement rectangle and trapezoid integration for v(t)=2*t and a curved function. Pass: compare to an analytic integral, halve the step and report how error changes.

Student file: `labs/student/lab_08_2.py`

Functions: `integrate`

Coding contract. Integrate callable f on finite endpoints a,b using n>=1 integer panels. method is exactly left or trapezoid. Return the signed result: reversing endpoints changes the sign; equal endpoints give zero. Reject unknown methods, bool n and invalid counts; sample values must be finite. The default is trapezoid. Do not use the endpoint value times the full interval as a general integral.

1. Implement integrate(f,a,b,n,method), supporting left and trapezoid rules with the documented signed-interval contract.
2. Compare t^2, 2t, and a chosen curved function against antiderivatives using n,2n,4n panels.
3. Include reversed limits, an equal-endpoint interval, invalid n, and a function returning a nonfinite value.
4. Write a displacement-versus-distance example and a normalized density calculation with units.

Check command: `python check_lab.py 08.2`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 08.3 — Newtonian mechanics and differential equations

Original brief: Implement a particle under constant acceleration and a damped spring. Pass: compare the constant-acceleration trajectory to its analytic solution and state what collision, rotation and air-resistance effects are omitted.

Student file: `labs/student/lab_08_3.py`

Functions: `constant_acceleration`

Coding contract. Return the pair (x0+v0*t+0.5*a*t*t, v0+a*t) for finite scalar x0,v0,a and nonnegative finite t. The formula assumes constant acceleration over the interval; no collision or drag is modeled. Reject invalid time. This analytic reference is not a numerical integrator.

1. Implement constant_acceleration(x0,v0,a,t), returning position and velocity with finite values and t>=0.
2. Use the spring model to run both damped and undamped examples. Record m,k,c,initial state, units, and omitted effects.
3. Derive the constant-acceleration solution and the continuous spring energy-rate identity on paper.
4. Compare the numerical undamped spring against its analytic sine/cosine trajectory at identical timestamps.

Check command: `python check_lab.py 08.3`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 08.4 — Time integration, stability and error

Original brief: Compare explicit and semi-implicit Euler on an undamped spring at three timesteps. Pass: plot or tabulate position and energy error, identify an unstable case, and avoid calling visual smoothness proof of physical accuracy.

Student file: `labs/student/lab_08_4.py`

Functions: `spring_step`

Coding contract. Use acceleration (-k*x-c*v)/m computed from the old state, with finite m>0, k>=0, c>=0 and dt>=0. method is explicit or semi. Explicit updates position with old velocity; semi updates velocity first and uses that new velocity for position. Return (x_next,v_next), reject unknown methods and invalid parameters. Zero timestep preserves state. The routine is a one-step model, not a stability guarantee.

1. Implement spring_step(x,v,mass,stiffness,damping,dt,method), supporting explicit and semi updates and rejecting invalid physical parameters.
2. Compare both methods at h=0.1,0.05,0.025 over the same two-second horizon, against the analytic undamped solution.
3. Add an unstable semi case h=0.25 for m=1,k=100 and a bounded run length. Preserve energy and error tables, not only an animation.
4. Derive the explicit energy-growth identity; explain at least one model error that timestep refinement cannot repair.

Check command: `python check_lab.py 08.4`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Module 08 — Fifteen written problems

Show intermediate reasoning, domains and units. Problems marked as proofs/derivations require an argument, not only an answer. Preserve wrong attempts and corrections.

08.P01 — calculation

Differentiate 4t^3-2t+7, then evaluate the derivative at t=2.

08.P02 — calculation

Differentiate sin(3t) and explain the extra factor.

08.P03 — units

For x(t)=t^3 meters, find velocity and acceleration at t=2 seconds.

08.P04 — calculation

Compute the forward difference of t^2 at t=3 with h=0.2 and its error versus the derivative.

08.P05 — derivation

Derive the centered-difference error for f(t)=t^3.

08.P06 — calculation

Integrate 3t^2 from 0 to 2.

08.P07 — quadrature

Use two-panel trapezoids to integrate t^2 on [0,1]. Compare to the exact result.

08.P08 — derivation

Derive the constant-acceleration position formula from dx/dt=v and dv/dt=a.

08.P09 — mechanics

A particle starts at x0=2 m with v0=3 m/s under a=-2 m/s^2. Find x and v at t=2 s.

08.P10 — units

A 4 kg body receives force 12 N for 0.5 s. Find acceleration, impulse and velocity change.

08.P11 — mechanics

For m=2,k=18,c=0, find omega and the continuous period.

08.P12 — integration

For x=1,v=2,a=-4 and h=0.1, compare explicit and semi-implicit one-step updates when acceleration is constant.

08.P13 — proof

Prove dE/dt=-c*v^2 for E=0.5*m*v^2+0.5*k*x^2 and the damped spring equations.

08.P14 — derivation

For an undamped spring, derive the explicit Euler one-step energy ratio.

08.P15 — derivation

At fixed T=N*h under constant acceleration, derive explicit Euler position error relative to the analytic solution.

Module 08 — Review quiz

Five questions, two points each. Answer without the key; suggested pass is 8/10 with reasoning.

08.R1. What is wrong with comparing 100 steps at two different h values as the same final-time experiment?

08.R2. A centered derivative estimate for |t| at zero is zero. Does that prove differentiability?

08.R3. Name three errors that reducing h may not fix.

08.R4. For m=1,k=100 and h=0.25, evaluate the stated semi-implicit oscillator condition.

08.R5. What should a simulator report after nonfinite values appear?

Integration project — Projectile and spring laboratory

Analytic equations, tested particle/spring functions, timestamped trajectory CSVs, and a fixed-horizon refinement report comparing explicit and semi-implicit Euler.

Pass evidence: Include h=0.1,0.05,0.025 at T=2 seconds for the m=1,k=100 undamped spring, plus a bounded unstable h=0.25 case. State maximum and final position errors, energy behavior, units, and omitted physics.

Reviewer's changed requirement: Double stiffness and recompute omega and the derived linear stability condition before rerunning. Explain why smoother output is not an accuracy argument.

Retain the artifact for Gate B; this work accumulates rather than being discarded after the module.

Module 09 — Probability, statistics and evidence

Prerequisites: Modules 06 and 08.2.

Lab 09.1 — Probability and conditional reasoning

Original brief: Simulate independent and dependent binary events. Build a small contingency table for a hypothetical bug detector. Pass: calculate both P(alert|bug) and P(bug|alert) and explain why they differ.

Student file: `labs/student/lab_09_1.py`

Functions: `posterior_bug`

Coding contract. Return prior*sensitivity / (prior*sensitivity+(1-prior)*false_positive). All three probabilities must be finite in [0,1]. Reject an exactly zero denominator because conditioning on an event with model probability zero is undefined. Reject booleans. Keep P(alert|bug) distinct from P(bug|alert).

1. Implement posterior_bug(prior,sensitivity,false_positive), validating probability ranges and nonzero alert probability.
2. Construct the full four-cell detector table, including true/false positives and negatives, before using the formula.
3. Simulate independent binary events and a copied-outcome process with the same marginal rates; compare their joint events.
4. Vary only the base rate over 0.001,0.01,0.1 and explain the different posterior probabilities without calling one detector universally accurate.

Check command: `python check_lab.py 09.1`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 09.2 — Random variables, distributions and sampling

Original brief: Implement a weighted reward sampler and compare observed frequencies to the intended distribution over several sample sizes. Pass: record seed, generator, counts and expected values; explain finite-sample variation.

Student file: `labs/student/lab_09_2.py`

Functions: `weighted_index`

Coding contract. Accept a nonempty list/tuple of finite nonnegative weights with positive finite total, and a finite u in [0,1). Return the category index using left-closed/right-open cumulative probability intervals. A zero-weight category is never selected. Boundary u=0.25 for weights [1,0,3] belongs to index 2. Random-number generation is deliberately outside this deterministic function.

1. Implement weighted_index(weights,u), keeping the random generator outside this deterministic function. Test boundaries, zero weights, invalid u, and invalid totals.
2. Simulate the three-category reward model at 100,1000,10000 draws and at least three recorded seeds.
3. Derive the expectation and variance by hand; compare empirical counts and means without demanding exact frequency matches.
4. Demonstrate how an extra draw changes a shared stream, then show two separate generator instances preserve the unrelated subsystem sequence.

Check command: `python check_lab.py 09.2`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 09.3 — Inference, uncertainty and benchmark design

Original brief: Compare two implementations using the same inputs, randomized test order and repeated runs. Pass: report median, a tail percentile, sample count and limitations; distinguish observations from causal claims.

Student file: `labs/student/lab_09_3.py`

Functions: `percentile`, `mean_interval`

Coding contract. percentile accepts nonempty finite samples and p in [0,1], sorts a copy, and uses nearest rank: for p>0 take sorted[ceil(p*n)-1], with p=0 defined as the minimum. mean_interval requires at least two finite observations and a strictly positive finite supplied critical value. Return (sample_mean,lower,upper) with width critical*s/sqrt(n), sample variance denominator n-1. Neither routine chooses or verifies an independence or distribution model.

1. Implement percentile(values,p) with the nearest-rank convention, and mean_interval(values,critical) using sample variance with denominator n-1.
2. Run the included benchmark runner or two independently implemented equivalent operations. Preserve randomized pair order, warmup, raw samples, and the workload.
3. Report mean, median, p95 or p99 with their convention and sample count, plus paired differences and clearly qualified uncertainty.
4. Analyze the synthetic tail example separately. Never relabel its chosen numbers as measured hardware performance.

Check command: `python check_lab.py 09.3`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 09.4 — Regression, correlation and ethical telemetry

Original brief: Fit a simple performance model using entity count and measured update time. Hold out some workloads. Pass: report prediction error and one confounder; write a short data-minimization plan for the experiment.

Student file: `labs/student/lab_09_4.py`

Functions: `rmse`

Coding contract. Return root-mean-square prediction error for equal-length, nonempty finite actual/predicted lists or tuples. Reject mismatches, empty observations and nonfinite inputs. RMSE has the same units as the outcome. It is not the same statistic as mean signed residual or R-squared.

1. Implement rmse(actual,predicted) for nonempty equal-length finite sequences. Reuse the independently tested line-fitting routine.
2. Fit update time versus entity count from declared local measurements; reserve at least one workload group before fitting. The included synthetic CSV is practice only.
3. Compare the fitted model to a training-mean baseline, preserve residuals, and distinguish interpolation from extrapolation.
4. Write a data-minimization and provenance note, identifying one possible confounder and which claims the experiment does not support.

Check command: `python check_lab.py 09.4`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Module 09 — Fifteen written problems

Show intermediate reasoning, domains and units. Problems marked as proofs/derivations require an argument, not only an answer. Preserve wrong attempts and corrections.

09.P01 — probability

Two independent events have probabilities 0.3 and 0.4. Find the probability both occur and the probability at least one occurs.

09.P02 — probability

A success probability is 0.1 on each of three independent trials. Find the probability of at least one success.

09.P03 — conditional

A table has 80 true alerts and 120 false alerts. There are 100 actual bugs. Compute precision-like P(bug|alert) and sensitivity.

09.P04 — Bayes

With prior=0.02, sensitivity=0.9 and false-positive rate=0.1, compute P(bug|alert).

09.P05 — derivation

Derive the detector Bayes formula from conditional probability and a partition into bug/no-bug.

09.P06 — distribution

A reward is 0 with probability 0.8 and 10 with probability 0.2. Find expectation and variance.

09.P07 — distribution

Find mean and variance of a binomial success count with n=20,p=0.25.

09.P08 — sampling

Partition [0,1) for weights [1,0,3]. Which category contains u=0.25?

09.P09 — proof

Prove E[X+Y]=E[X]+E[Y] for a finite joint distribution without assuming independence.

09.P10 — statistics

For samples [1,2,3,4,100], find mean, median and nearest-rank p80 and p95.

09.P11 — statistics

For [2,2,4,4], compute sample mean, sample variance and standard error of the mean.

09.P12 — interpretation

Explain the 95% frequentist confidence-interval interpretation without assigning probability to a fixed realized parameter.

09.P13 — experiment

A benchmark runs A only on a cool machine and B only after a long heavy workload. What is confounded, and how could the design improve?

09.P14 — regression

Fit y=a*x+b to points (0,1),(1,3),(2,5). Predict at x=3; the held-out observed value is 8. Give the residual.

09.P15 — derivation

Show that variance of the average of n independent identically distributed variables with finite variance s^2 is s^2/n.

Module 09 — Review quiz

Five questions, two points each. Answer without the key; suggested pass is 8/10 with reasoning.

09.R1. A detector is 90% sensitive. Is an alert a 90% chance of a bug?

09.R2. What changes when you add an extra draw to a shared pseudorandom stream?

09.R3. Why is a tiny p-value not sufficient evidence that a player will notice an improvement?

09.R4. What is wrong with treating synthetic frame data as a hardware benchmark?

09.R5. Name the minimum evidence for your own comparison experiment.

Integration project — Randomness and performance evidence

A detector contingency table, reward-distribution experiment with recorded seeds, and one locally measured comparison with randomized paired ordering, raw results, descriptive summaries, qualified uncertainty, and a held-out prediction exercise.

Pass evidence: Synthetic and measured datasets remain separately labeled. Both algorithms produce equivalent answers. Report workload, machine/software boundary, warmup, trials, inclusion rules, percentile convention, and independence limitations. A null or adverse result is acceptable.

Reviewer's changed requirement: Change the detector base rate and the benchmark workload family. Explain which conclusions can transfer and which need new evidence.

Retain the artifact for Gate B; this work accumulates rather than being discarded after the module.

Module 10 — Signals, optimization and numerical foundations

Prerequisites: Modules 07-09.

Lab 10.1 — Multivariable calculus and gradients

Original brief: Derive gradients for three functions and check them with centered differences. Pass: test nonzero and stationary points, vary perturbation size and report errors rather than claiming exact equality.

Student file: `labs/student/lab_10_1.py`

Functions: `gradient`

Coding contract. Accept a callable that maps a finite point vector to a finite scalar, a nonempty point list/tuple, and positive finite perturbation h. Compute each partial derivative by a centered difference, leaving other coordinates unchanged. Return a tuple. Reject an unresolvable perturbation or nonfinite function result. This check approximates derivatives; it does not symbolically prove them.

1. Implement gradient(f,point,h), perturbing one coordinate at a time, rejecting invalid/unresolvable h and nonfinite evaluations.
2. Derive and check three gradients, including a coupled function such as x*y+x^2 and a function with a stationary saddle.
3. Derive the Jacobian example and compare predicted versus exact finite changes over several step magnitudes.
4. Create a y-up terrain-normal calculation and explicitly document which variables parameterize the surface.

Check command: `python check_lab.py 10.1`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 10.2 — Sampling, aliasing and Fourier ideas

Original brief: Generate low- and high-frequency sine samples, then downsample a checker pattern with and without prefiltering. Pass: explain the artifacts using frequency content rather than calling them random noise.

Student file: `labs/student/lab_10_2.py`

Functions: `alias_frequency`

Coding contract. Return the nonnegative baseband alias magnitude in [0,fs/2] for finite frequency>=0 and finite sample_rate>0. First fold frequency modulo fs, then mirror above fs/2. This

result alone does not specify phase or the sign relation of sampled sine waves. Reject invalid domains; no waveform reconstruction is performed.

1. Implement alias_frequency(frequency,sample_rate), returning only the nonnegative baseband magnitude and documenting that phase is not encoded.
2. Generate the 900/100 Hz cosine pair and the corresponding signed sine relation at fs=1000. Verify both analytically and numerically.
3. Downsample a checker pattern with both starting phases and with a declared prefilter. Do not hide the filter delay or boundary behavior.
4. Compute the four-point DFT example by hand and explain why this is a different issue from the alias demonstration.

Check command: `python check_lab.py 10.2`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 10.3 — Convolution, filters and systems

Original brief: Implement a one-dimensional convolution with documented boundary handling. Test an impulse, a constant and a step. Pass: predict the output before running and identify the filter's delay convention.

Student file: `labs/student/lab_10_3.py`

Functions: `convolve`

Coding contract. Return full linear convolution with zero extension: y[n]=sum_k kernel[k]*signal[n-k]. Nonempty inputs give len(signal)+len(kernel)-1 outputs. An empty signal returns []; an empty kernel is rejected. Both inputs are list/tuple sequences of finite values. The full convolution alignment must be tested with an asymmetric kernel to distinguish it from correlation.

1. Implement convolve(signal,kernel) as full zero-extended convolution. Return [] for empty signal, reject empty kernel and nonfinite data.
2. Predict and test impulse, constant, step, and asymmetric-kernel outputs. Include output length and alignment checks.
3. Compare full output with a causal same-length prefix and with a separately specified boundary alternative.
4. Explain gain from sum(kernel), the filter delay in samples and seconds, and why a future-looking centered filter is different in real time.

Check command: `python check_lab.py 10.3`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Lab 10.4 — Optimization, conditioning and approximation budgets

Original brief: Implement gradient descent on a quadratic and a two-variable problem. Pass: compare step sizes, record residuals and state a stopping rule; complete Gate B with no unsupported claim of global optimality.

Student file: `labs/student/lab_10_4.py`

Functions: `descent_quadratic`

Coding contract. Minimize only the stated scalar objective (x-3)^2 from finite x0 using finite alpha>0, integer max_steps>=0 and finite tol>0. Stop when absolute gradient<=tol or the step budget is exhausted. Return x, value, gradient, iterations, converged, reason and history. The initial point counts as zero updates. A nonconverged result must say iteration_limit. Reject invalid parameters and do not conceal nonfinite arithmetic.

1. Implement descent_quadratic(x0,alpha,max_steps,tol) with explicit convergence/iteration-limit status, achieved gradient, and a history of iterates.
2. Compare alpha=0.25,0.5,1,1.1 from the same start. Preserve nonconvergent cases and verify reported status at the final allowed iteration.
3. Implement a two-variable extension and derive its step-size restriction from the two error factors, rather than guessing by trial alone.
4. Complete Gate B: assemble proofs, tested geometry, dynamics/refinement, sampling/filtering, probability, measured experiment, and a reproducibility handoff.

Check command: `python check_lab.py 10.4`. Reference comparison: add `--reference` after an independent attempt.

Evidence to retain: prediction, derivation/assumptions, code revision, exact command, observed values with units, at least one failed case and correction, and an independent transfer response. Add five new edge/failure cases across this module beyond the worked examples.

Module 10 — Fifteen written problems

Show intermediate reasoning, domains and units. Problems marked as proofs/derivations require an argument, not only an answer. Preserve wrong attempts and corrections.

10.P01 — gradient

Find the gradient of f(x,y)=x^2+xy+2y^2 and evaluate it at (1,2).

10.P02 — Jacobian

For F(x,y)=(x+y,x*y), find the Jacobian at (2,3) and its prediction for delta=(0.01,0).

10.P03 — derivation

Let g(z)=z1^2+z2^2 and F(x,y)=(2x,y). Derive grad(g(F(x,y))) directly and through the Jacobian chain rule.

10.P04 — optimization

Classify the origin for x^2+y^2, -(x^2+y^2), and x^2-y^2.

10.P05 — sampling

Find baseband alias magnitudes of 100,700,1100 Hz sampled at 1000 Hz.

10.P06 — proof

Prove that zero-phase cosines at f and fs-f have identical samples at rate fs.

10.P07 — sampling

A 1,000 Hz sampled signal is reduced by a factor of four. State the new Nyquist boundary and which original content needs suppression before decimation.

10.P08 — Fourier

Compute the unnormalized four-point DFT of [1,1,1,1].

10.P09 — convolution

Compute full zero-extended convolution of [2,0,1] with [1,2].

10.P10 — proof

Prove convolution by a fixed kernel is linear.

10.P11 — filter

A causal three-tap average is applied to a constant finite input of six ones with zero extension. Give the full output.

10.P12 — filter

What is the group delay of a symmetric seven-tap causal FIR at 2000 samples/s where its response is nonzero?

10.P13 — optimization

For f=(x-3)^2, x0=0 and alpha=0.5, compute the next point and gradient.

10.P14 — proof

Prove the scalar quadratic convergence interval for fixed-step gradient descent.

10.P15 — error budget

Two deterministic absolute error bounds are 0.02 and 0.03 in the same output units. Give a guaranteed combined bound and explain why quadrature is not automatic.

Module 10 — Review quiz

Five questions, two points each. Answer without the key; suggested pass is 8/10 with reasoning.

10.R1. What changes in a gradient when parameters use different unit scales?

10.R2. Why cannot a filter always repair aliasing after sampling?

10.R3. What tests expose an incorrect convolution alignment?

10.R4. A solver stops at its iteration cap with a nonzero residual. What should the record say?

10.R5. What makes Gate B more than a folder of plots?

Integration project — Gate B: integrated game-mathematics notebook

The accumulated projects plus a sampling/filtering demonstration, a gradient check, and a convergent/nonconvergent optimization comparison, assembled into one reproducible handoff.

Pass evidence: Satisfy CAPSTONE_SPEC.pdf: proofs, tested transforms, analytic dynamics/refinement, aliasing/filtering, probability checks, a measured experiment, data provenance, and an independent reproduction. These reuse module work; they are not five extra unrelated capstones.

Reviewer's changed requirement: Defend a changed coordinate convention, timestep, and misleading average. Reproduce a central result from a fresh folder and explain a known failure case.

Retain the artifact for Gate B; this work accumulates rather than being discarded after the module.